Rozdział 1

Test

1.1 Data conversion with LLMs

Text-First Extraction with Normalization decomposes the data-to-triples problem into two independent, fully batched LLM stages and a normalization stage. The method deliberately avoids any inter-instance coupling during extraction: each reference and each triple set is produced independently and in parallel, which makes the method stateless and easy to reason about, at the cost of not enforcing predicate consistency during extraction.

1.1.1 Text-First Extraction with Normalization

Stage 1 — Reference text generation. Let $D = \{d_1, \ldots, d_N\}$ denote the set of N raw structured instances obtained from the input JSON after instance extraction. In the first stage the model is invoked once per instance under a text-generation system prompt, P_{txt}, whose objective is to convert one structured data instance into concise natural language and to return the result as a JSON object of schema {`"text": "..."`}. The prompt further instructs the model to capture the most important information, trends, extremes, and notable conditions, and to summarize time series (e.g., weather forecasts) at a high level rather than listing every data point.

All N requests are packaged as a single OpenAI Batch API job and a single request outputs t_i, where t_i is the generated reference text for instance d_i. This batched stage yields, for every instance, a natural-language reference t_i that constitutes the intermediate representation from which triples are subsequently derived.

Stage 2 — Triples-from-text extraction. A second OpenAI Batch is then assembled in which every reference t_i is paired with a triples-from-text system prompt, P_{tr}, whose objective is to extract semantic triples from natural language text, returning JSON of schema {`"triples":["subject":"...","predicate":"...","object":"...`

`]`}. The prompt imposes two light constraints: it instructs the model to preserve the full meaning of the input text and to keep predicates in `lower_snake_case` when possible, and to avoid duplicate triples. Each instance is therefore an independent text-to-triples problem, and the batch can be fully parallelized. The output of a single request is T_i, where T_i is a set of triples that describe the reference text t_i.

Stage 3 — Post-hoc predicate normalization. Because Stage 2 imposes no inter-instance vocabulary control, this stage invokes a predicate-normalization routine. The routine operates on the flattened list of triples and iteratively merges semantically equivalent predicates:

Let Π be the set of distinct predicates present in the extracted triples. A Union-Find structure over Π is initialized with every predicate in its own singleton class.

For each round $r = 1, 2, \ldots$ (bounded by $2|\Pi|$):

- The model is prompted with the current set of canonical predicates (class representatives) and asked to propose candidate equivalent pairs under a candidate-pairs system prompt.
- If no candidate pairs are returned, or if the candidate set repeats a previously seen round signature, the loop terminates.
- Otherwise, every candidate pair is sent to the model under a strict equivalence system prompt that returns {`equivalent, confidence, reason`} and is instructed to judge equivalence strictly (mergeable in a knowledge graph), not mere relatedness.
- Pairs judged equivalent are merged in the Union-Find structure. If a round produces no merges, the loop terminates.

After termination, each predicate is mapped to the canonical representative of its class, defined as the first member of the class (deterministic, order-based choice). The triples are then rewritten with these canonical predicates.

In the normalization step, the final output combines: (i) generated references, (ii) extracted triples, and (iii) normalized triples alongside the canonical mapping and the comparison audit trail.

1.1.2 Rules-Iterative Refinement

Rules-Iterative Refinement introduces an explicit notion of extraction rules that act as an intermediate, human-inspectable artifact between the domain declaration and the final triple extraction. The rules are generated top-down from the domain name, then iteratively validated and revised against the generated references until they admit triple extraction for every instance. Only then is a single batched triple-extraction run executed under the finalized rules.

Stage 1 — Initial rule generation. Given the user-supplied problem domain string δ (e.g., weather forecast), the model is invoked once under a rule-creation system prompt, P_{rules}, and asked to design extraction rules for a domain. Its response is constrained by a strict JSON schema to contain exactly four fields:

$$R = \Big(\Pi_R,\ E,\ T_{ex},\ G\Big),$$

where Π_R is a list of predicate labels, E is a list of example triples $\{(\sigma_k, \pi_k, \omega_k)\}$ using those predicates, T_{ex} is a short natural-language passage consistent with the example triples, and G is a list of free-text transformation guidelines. The response is sanitized, which deduplicates predicates and guidelines, discards malformed example triples, and falls back to previous values for any empty field. The resulting R_0 constitutes the initial ruleset. Conceptually, R_0 is a self-contained specification of how the model believes instances of the domain should be verbalized and triplelized, complete with concrete formatting conventions embedded in the example triples.

Stage 2 — Reference text generation. The reference-generation batch is identical to that of the previous method: every instance d_i is sent to the text-generation system prompt P_{txt} within a single OpenAI Batch, producing references t_i.

Stage 3 — Rule validation and iterative revision. The core of the method is an iterative loop, bounded by $R_{\max}$. The loop maintains a current ruleset R and operates as follows for each round $r = 1, \ldots, R_{\max}$:

1. **Feasibility scan.** A random reference text t_i is selected and the model is invoked under a rule-feasibility system prompt, P_{feas}, which receives the current rules R and the text t_i, and

returns {`possible, reason, rule_gaps`}, where `rule_gaps` is a list of textual descriptions of what is missing from R in order to extract triples from t_i.

2. **Pass condition.** If the scan reaches the $R_{\max}$ without encountering a `possible = false` judgment, the rules are declared validated against all texts and the loop terminates.

3. **Failure and revision.** As soon as a reference t_{i^*} is judged infeasible under R, the scan halts. The revision system prompt, P_{rev}, is then invoked with the domain δ, the current rules R, the failing text t_{i^*}, the failure reason, and the enumerated `rule_gaps`. Its response is again constrained to the four-field rules schema. The model is instructed to keep existing high-quality rules, but add or adjust what is necessary to cover the failing case. The sanitized result replaces R as the new current ruleset.

4. **Restart.** Because revising the rules may break previously validated instances or expose new gaps, the scan is restarted and the round number r set to 1. The loop thus monotonically accumulates coverage, but backtracks the scan position whenever the rules are modified.

If the budget $R_{\max}$ is exhausted before all texts are validated, the method emits a warning and proceeds to final extraction with the best-effort ruleset; the output records that validation was incomplete.

This stage implements a generate-test-revise loop in which the rules are the artifact being refined, the reference texts act as test cases, and the feasibility prompt acts as a model-internal test oracle.

Stage 4 — Final batched extraction under finalized rules. Once the validation loop has terminated, a single OpenAI Batch is submitted in which every reference t_i is paired with a triples-from-text-with-rules system prompt, $P_{tr\&R}$. This prompt instructs the model to extract semantic triples from the natural language text using the provided rules, to follow the rule guidelines, to mimic the formatting style of the example triples, and to prefer predicates from the rule predicate list whenever possible. The rules R are serialized and injected in each request. The batch output is parsed into T_i.

1.1.3 Evolving Catalog

Evolving Catalog is a hybrid text-first data-to-triples pipeline that combines an evolving predicate catalog with a stability-triggered transition from sequential to batch processing. It is designed to keep the predicate vocabulary consistent across instances while reducing the latency overhead of per-instance, order-dependent LLM calls.

Stage 1 — Reference text generation. An OpenAI Batch is assembled in which every instance d_i is paired with the text-generation system prompt P_{txt}, whose purpose is to produce a concise natural-language summary that captures the most important information, trends, extremes, and notable conditions, and summarizes time series at a high level. The batch output is parsed into text reference t_i.

Stage 2 — Sequential catalog-driven triple extraction. The second stage maintains a mutable predicate catalog C, initialized as empty. The catalog is a set of entries $\{(\pi, e_\pi)\}$, where each entry associates a predicate label π with an example triple $e_\pi = (s, \pi, o)$ in which that predicate was first observed. The catalog thus carries both a normalized vocabulary and a small set of usage examples that serve as in-context guidance for the extraction model.

The instances are processed strictly in order:

1. **Bootstrap instance ($i = 0$).** Because C is empty, the reference t_0 is sent to the model under a first-instance system prompt, P_{first}, that simply asks for semantic triples from the text with concise `lower_snake_case` predicates and no duplicates. This yields a set of triples T_0.

2. **Catalog-guided instances ($i \geq 1$).** For each subsequent instance d_i, the reference t_i is sent under a catalog-guided system prompt, P_{cat}, in which the current catalog C is serialized and injected into the user prompt. The model is instructed to use only predicates from the provided catalog when possible and to introduce a new predicate only when none of the catalog predicates can describe the fact. This yields triples T_i.

   After each instance, every triple in T_i is inspected. If a triple contains a predicate $\pi \notin C$, a new entry (π, e_π) is added to C, where e_π is set to that triple (the first occurrence).

Stage 3 — Stable-window heuristic and batched tail processing. A counter w tracks the number of consecutive instances that did not introduce any new predicate; it is reset to zero whenever a new predicate is added. Let W denote the stable window parameter. The sequential phase terminates as soon as $w \geq W$ and at least one instance remains unprocessed. Let i^* be the index at which this condition is first met; the catalog C at that point is declared frozen.

All instances $d_{i^*+1}, \ldots, d_N$ (the tail) are then processed in a single second OpenAI Batch, again under P_{cat} and with the frozen catalog C injected into every request. Two design choices are important here:

- The tail is parallelized, hence the cost of the remaining $N - i^* - 1$ LLM calls is amortized into one batch.

- Predicates that first appear within the tail are not added back to C. The catalog is finalized at the moment of stability, which guarantees that the predicate vocabulary presented to a human evaluator is fully determined by the sequential phase and does not silently grow during batch processing.